MASTER OF SCIENCE
IN ENGINEERING

Recurrent Neural Networks

Part 2: Generative models

Jean Hennebert
Martin Melchior

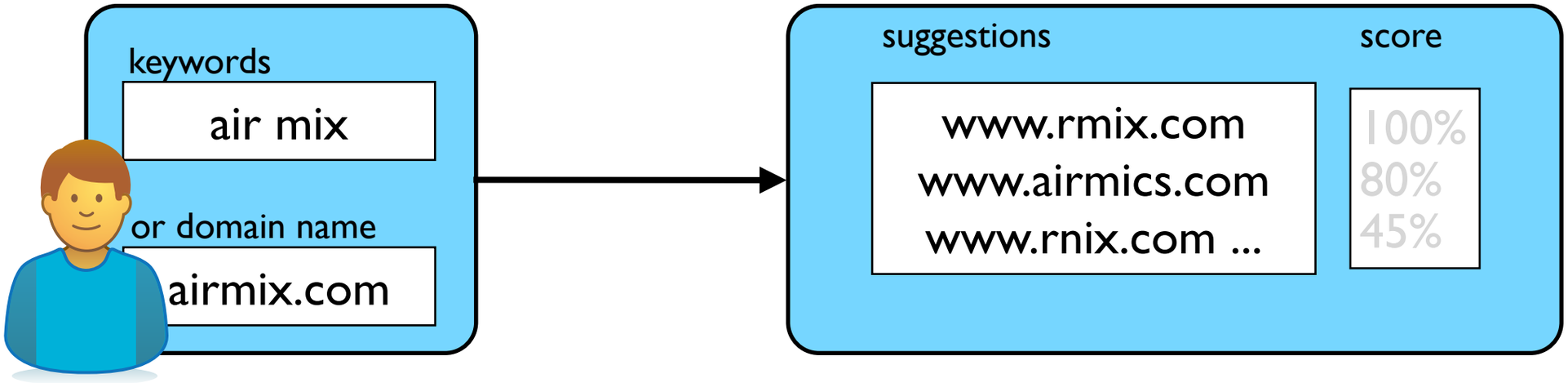
Generative RNNs

- use case

Domain name suggestion

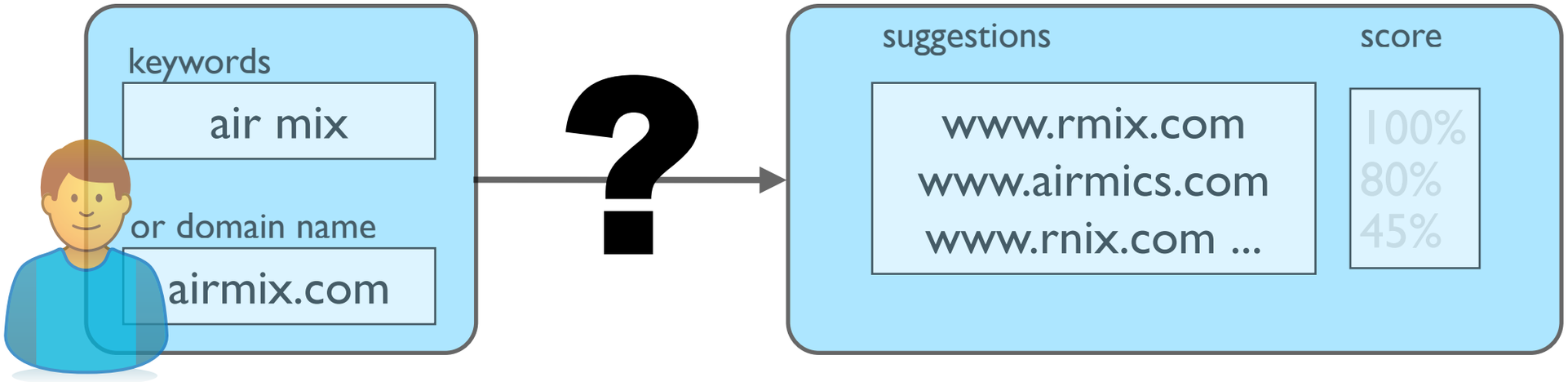
VERISIGN

Similar sounding domain name suggestion



- airmix.com is taken. I would like to know what are the similar sounding domain names that are available.
- I decide to book rmix.com that shows a 100% phonetic matching with what I initially wanted.

Similar sounding domain name suggestion

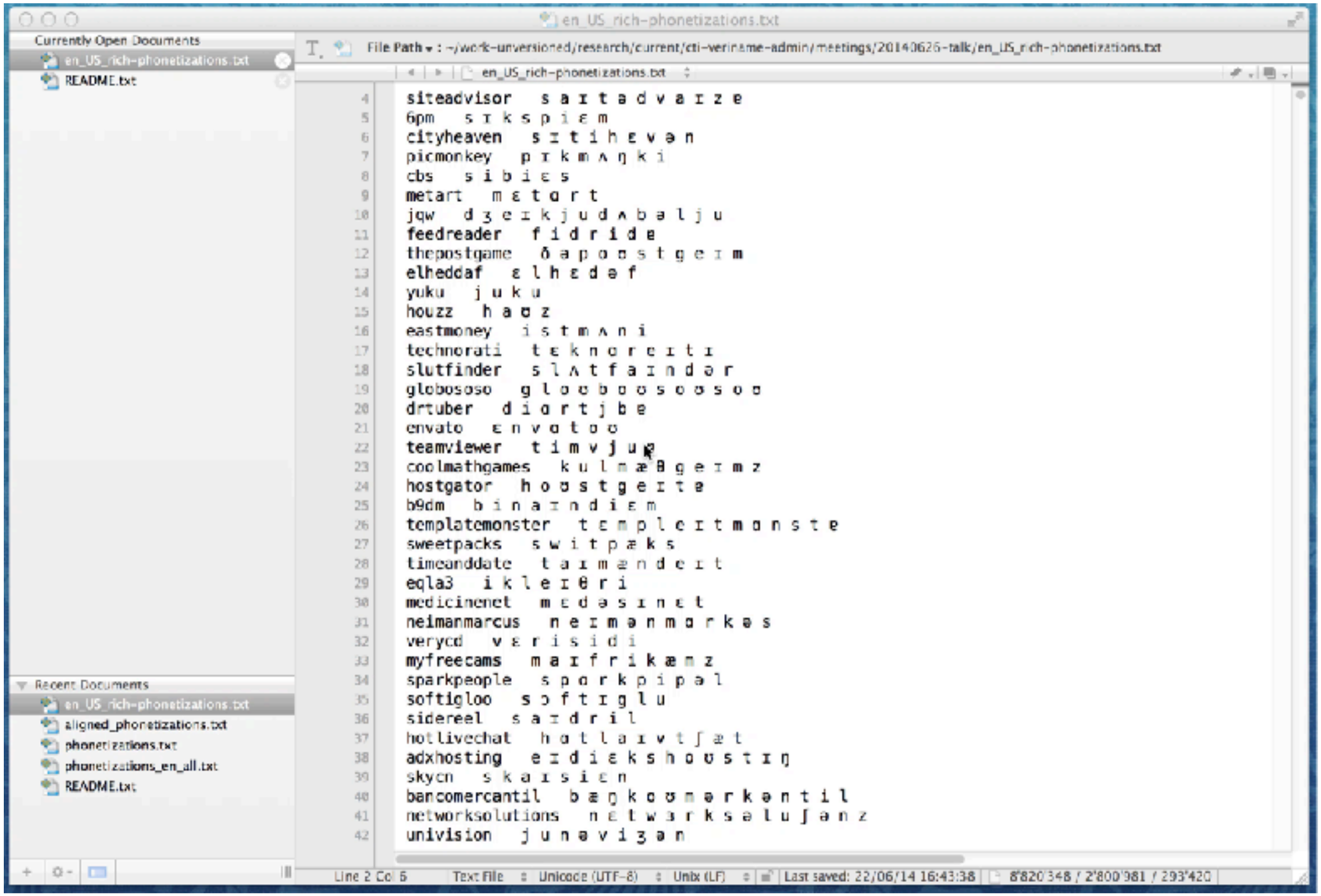


- Challenges:
  1. Transform input into phoneme sequences

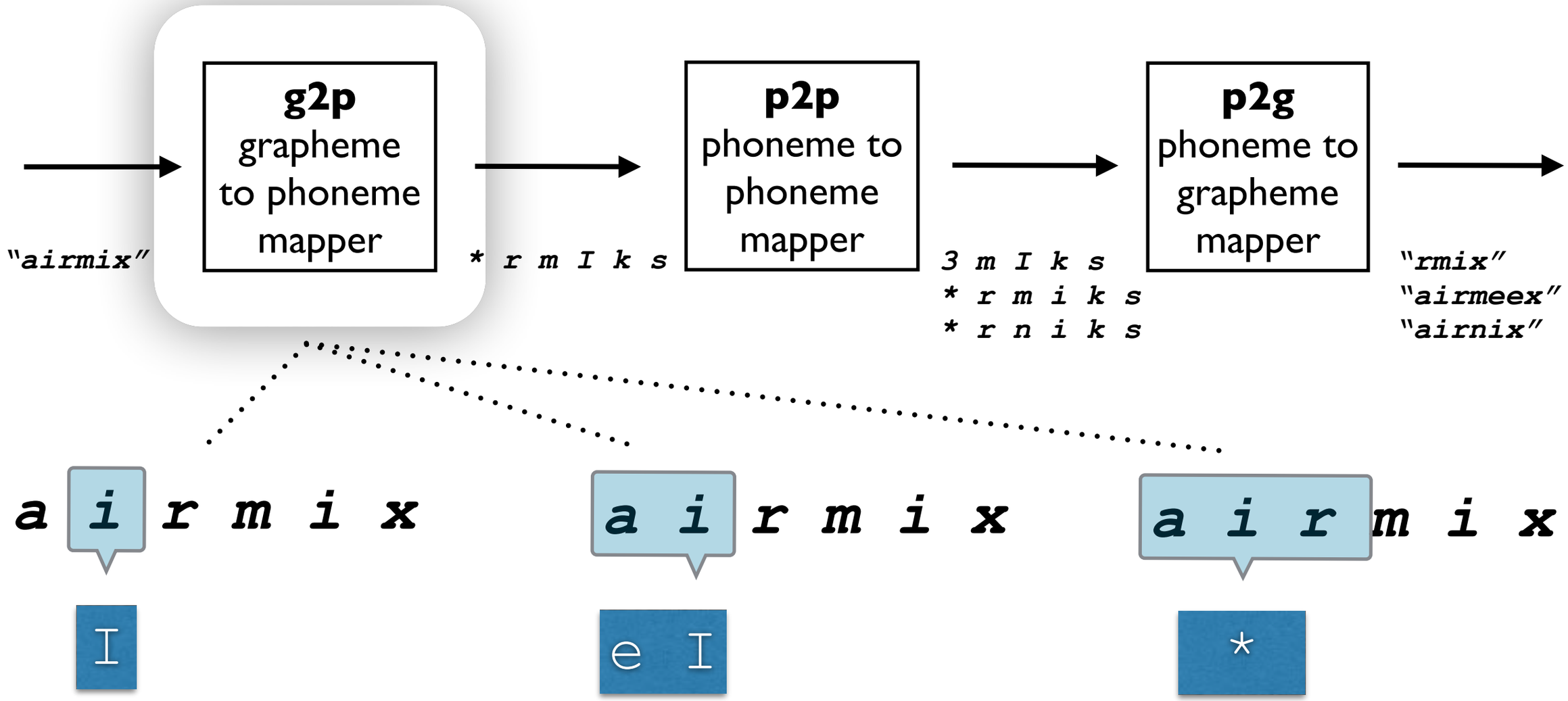
     invented words, brand names, family names, etc.
  2. Find the phonetic patterns
  3. Replace phonetic patterns with similar sounding ones
  4. Transform back into sequence of characters
- Constraints:
  1. Scale to as many language as possible !
  2. The algorithms should be efficient if we want to deploy !

What Languages Sound Like To Foreigners - http://www.youtube.com/watch?v=ybcvlxivscw

Phonetic structures

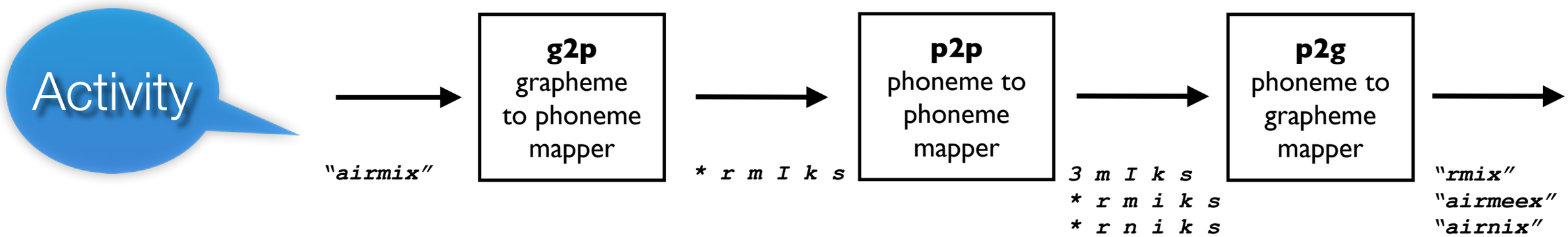


Similar sounding domain name suggestion

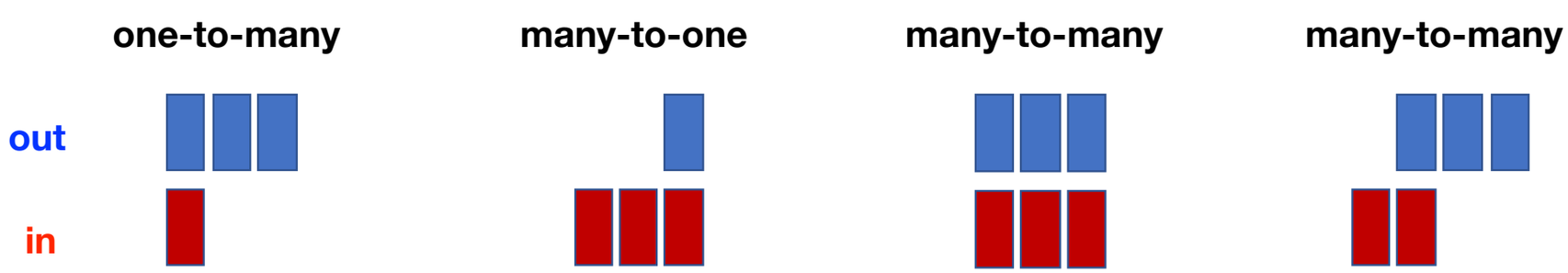


Different g2p mapping according to g context

RNN strategies

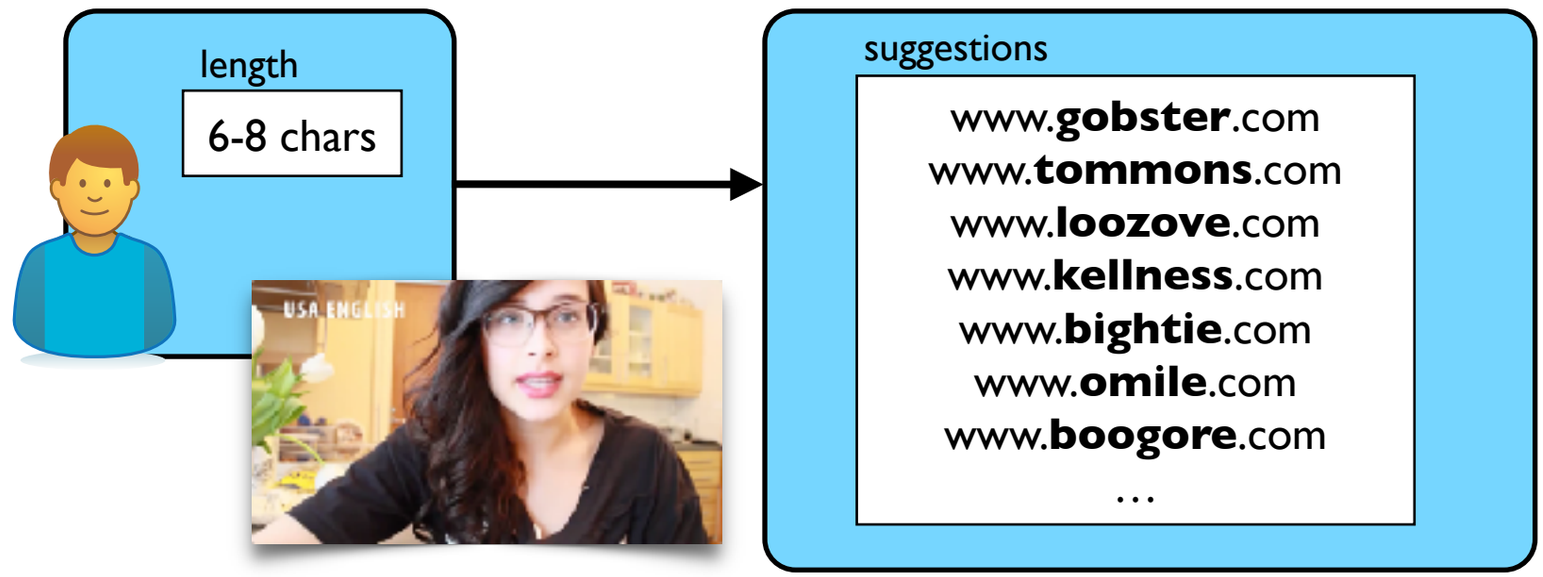


- For each block, which RNN strategy would you use?

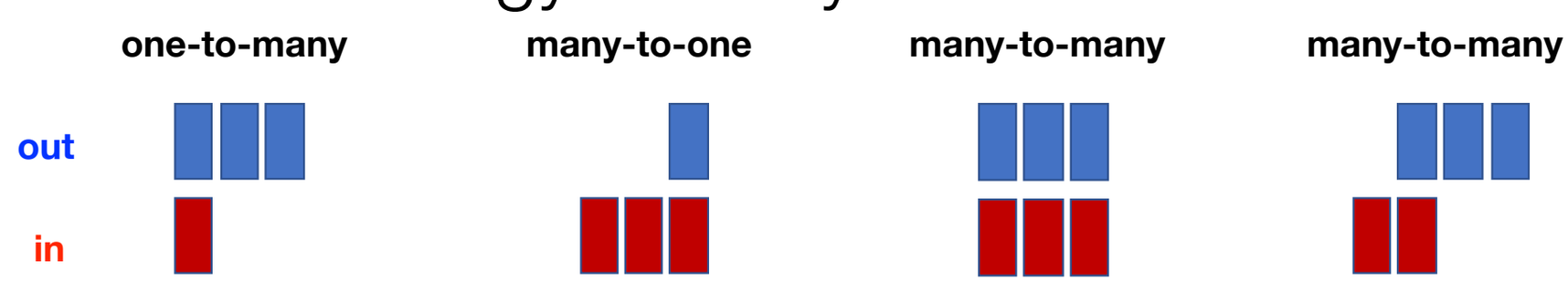


Random generation of (good) sounding domain names

Activity

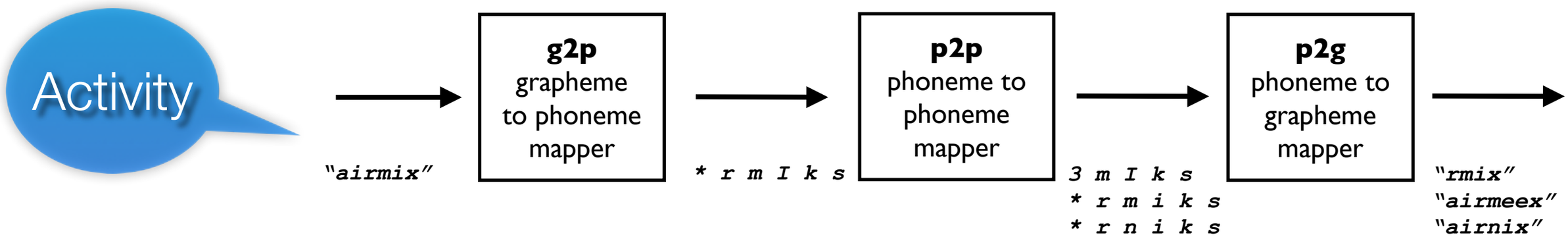


- Which RNN strategy would you use?



- Which model to use? (SimpleRNN, LSTM, GRU)

Supported Languages



- We encountered no big difficulties for English, French, German, Spanish, Portuguese, Italian, Russian
- We had more difficulties (or no solutions) for Korean, Japanese, Arabic, Chinese
- Why?

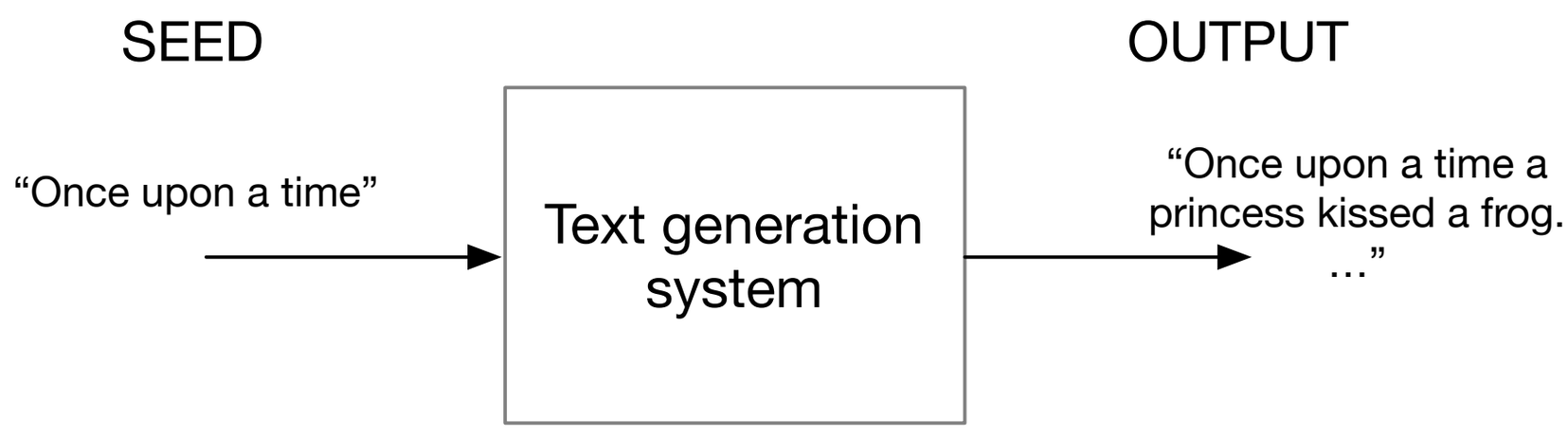
Generative RNNs

- Principles
- Two approaches
- Many to one
  - Example on text generation
- Many to many
  - Time distributed back-propagation
  - Example on text generation

Principle of data generation

A **generative system** is a system able to generate new consistent data from a **seed**. By consistent, we mean respecting temporal or spatial “structures” that have been learned from the input space.

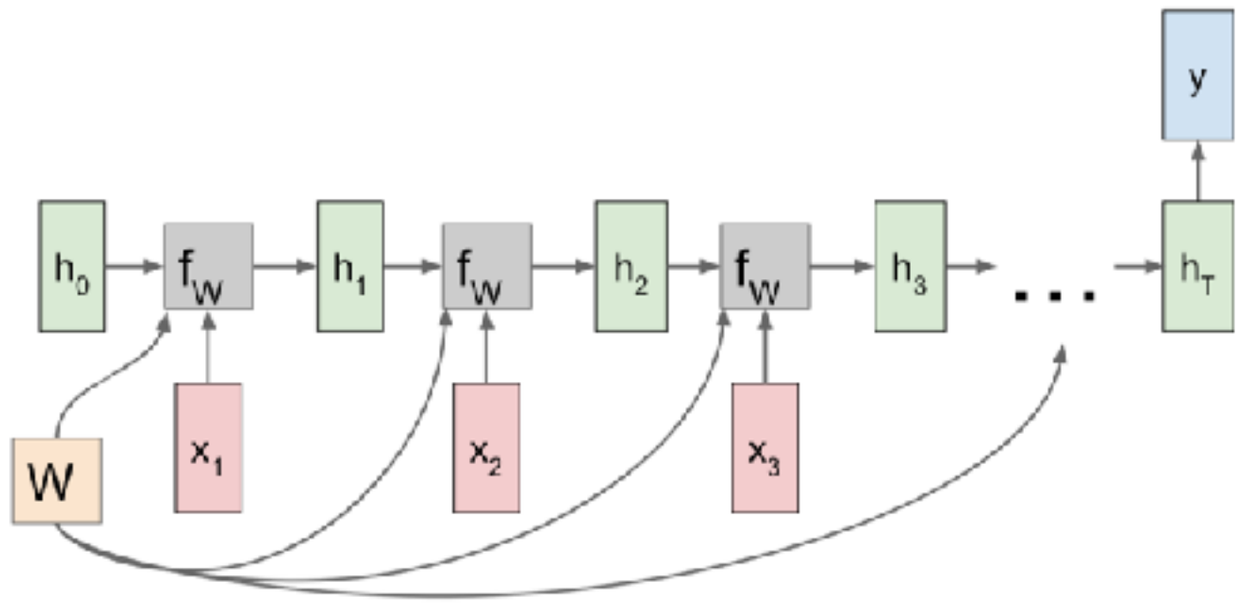
- Here we deal with *sequential* data that can be related to space (image), time (speech, music) or sequence of symbols (text, DNA).



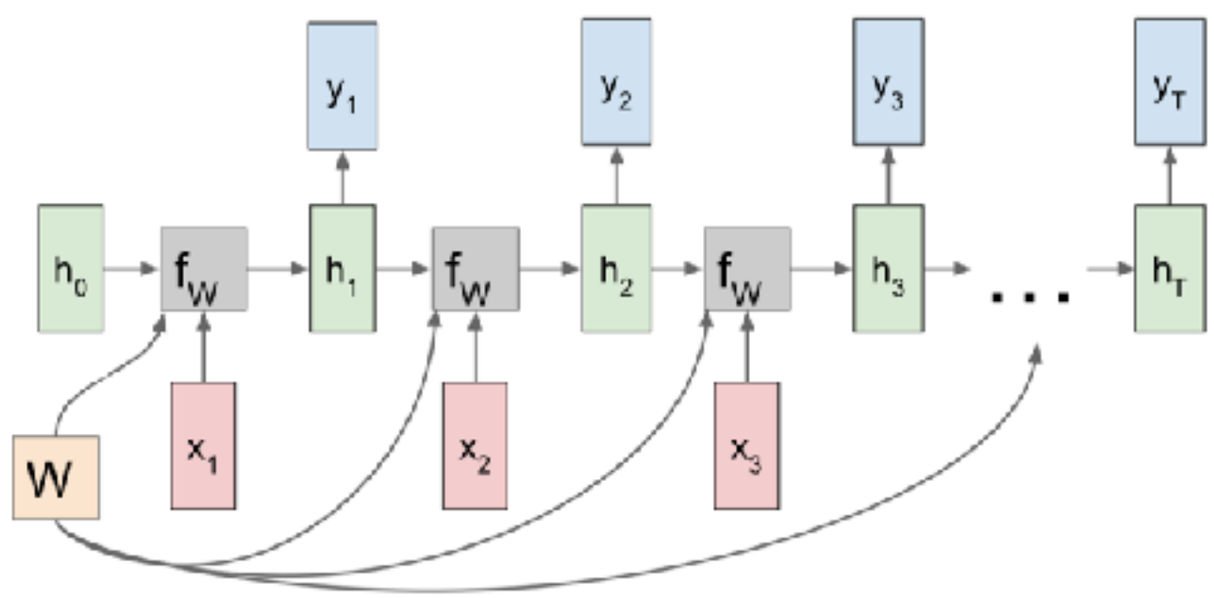
How to generate new sequential data with RNNs ?

- 2 approaches

Many to one

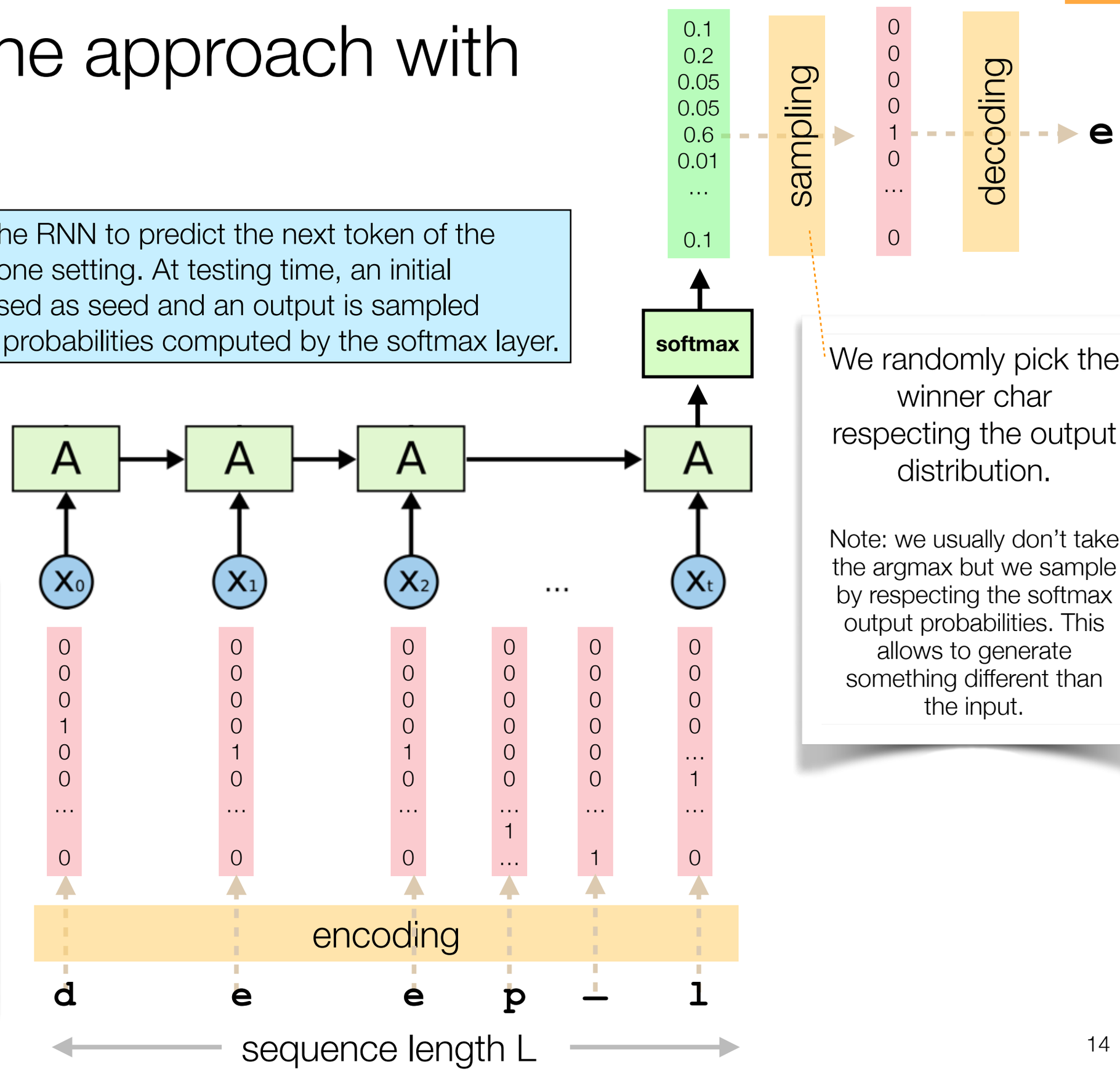


Many to many



Many to one approach with RNNs

The principle is to train the RNN to predict the next token of the sequence in a many-to-one setting. At testing time, an initial sequence of tokens is used as seed and an output is sampled respecting the posterior probabilities computed by the softmax layer.



We randomly pick the winner char respecting the output distribution.

Note: we usually don't take the argmax but we sample by respecting the softmax output probabilities. This allows to generate something different than the input.

One-hot vector with a '1' at the position according to the position of the letter in the alphabet size D

```
{'a', 'b', 'c',…,
 'A', 'B', 'C',…,
 '-', "'",
'_','END'}
```

Example on Shakespeare texts

- Training set:
  - 1’115’394 characters of Shakespeare.
  - 65 different chars including lower-upper cases, space, punctuation, carriage returns, etc.

shakespeare.txt

```
First Citizen:
Before we proceed any further, hear me speak.

All:
Speak, speak.

First Citizen:
You are all resolved rather to die than to famish?

All:
Resolved. resolved.

First Citizen:
First, you know Caius Marcius is chief enemy to the people.

All:
We know't, we know't.

First Citizen:
Let us kill him, and we'll have corn at our own price.
Is't a verdict?

All:
No more talking on't; let it be done: away, away!

Second Citizen:
One word, good citizens.

First Citizen:
We are accounted poor citizens, the patricians good.
What authority surfeits on would relieve us: if they
would yield us but the superfluity, while it were
wholesome, we might guess they relieved us humanely;
but they think we are too dear: the leanness that
afflicts us, the object of our misery, is as an
inventory to particularise their abundance; our
sufferance is a gain to them Let us revenge this with
our pikes, ere we become rakes: for the gods know I
speak this in hunger for bread, not in thirst for revenge.

Second Citizen:
Would you proceed especially against Caius Marcius?

All:
Against him first: he's a very dog to the commonalty.
```

Preliminary discussion - Shakespeare sonnets

Activity

- What can we expect out of the RNN?
  - Are the generated phrases going to “make sense”?
  - Are the words going to be valid words?
- How can we evaluate a generator?
  - At training time
  - At testing time

Define the many to one model in `Keras`

The model is very similar to what is used for the name classification. The only difference is that here we have D=65 different chars as output (`num_char`).

SimpleRNN layer:

- **`units`:** specifies the dimension of the hidden state variable.
- **`return_sequences=False`** means that no sequence is returned, i.e. just the hidden state at the last step.
- **`input_shape`:** length of input sequence (`sentence_length`) and dimension of an element of the input sequence (`num_char`)

We add a Dense and softmax activation layers for the binary classification.

```
# Define our model
model = keras.models.Sequential()
model.add(keras.layers.SimpleRNN(256, input_shape=(sentence_length, num_chars),
                                 return_sequences=False))
model.add(keras.layers.Dense(num_chars))
model.add(keras.layers.Activation('softmax'))
model.compile(loss='categorical_crossentropy', optimizer='adam',
              metrics=['accuracy'])
model.summary()
```

```
Model: "sequential"
_________________________________________________________________
Layer (type)                 Output Shape              Param #
=================================================================
simple_rnn (SimpleRNN)       (None, 256)               82432
_________________________________________________________________
dense (Dense)                (None, 65)                16705
_________________________________________________________________
activation (Activation)      (None, 65)                0
=================================================================
Total params: 99,137
Trainable params: 99,137
Non-trainable params: 0
_________________________________________________________________
```

$$\left.\begin{aligned} \mathbf{w}_h &: 256 \times 256 = 65536 \\ \mathbf{w}_x &: 256 \times 65 = 16640 \\ \mathbf{b}_h &: 256 \times 1 = 256 \end{aligned}\right\} \quad 82432$$

`# unique char = 65`

$$256 \times 65 + 65 = 16705$$

Load the data

The corpus has a sequence length of 1'115'394 chars.

```
import numpy as np
import random
import matplotlib.pyplot as plt
import tensorflow as tf

keras = tf.keras

# wget the file from https://storage.googleapis.com/download.tensorflow.org/
data/shakespeare.txt
with open("shakespeare.txt") as corpus_file:
    corpus = corpus_file.read()
    corpus_length = len(corpus)
print("Loaded a corpus of {0} characters".format(corpus_length))
```

```
Loaded a corpus of 1115394 characters
```

Get a list of unique chars : 65

Define en encoding (char to int) and decoding dictionary (int to char).

```
# Get a unique identifier for each char in the corpus,
# then make some dicts to ease encoding and decoding
chars = sorted(list(set(corpus)))
num_chars = len(chars)
encoding = {c: i for i, c in enumerate(chars)}
decoding = {i: c for i, c in enumerate(chars)}
print("Our corpus contains {0} unique characters.".format(num_chars))
```

```
Our corpus contains 65 unique characters.
```

Prepare the inputs and outputs

`sentence_length` is the length of the sequence and `skip` defines the stride we do on the input sequence

Chop the corpus into sentences of length 20. Take the 21st char as target.

A given input sequence is encoded into the corresponding char index defined in the encoding dictionary.

An example of x and y.

```
# chop up our data into X and y, slice into roughly
# (num_chars / skip) overlapping 'sentences' of length
# sentence_length, and encode the chars
sentence_length = 20
skip = 1
X_data = []
y_data = []
for i in range (0, len(corpus) - sentence_length, skip):
    sentence = corpus[i:i + sentence_length]
    next_char = corpus[i + sentence_length]
    X_data.append([encoding[char] for char in sentence])
    y_data.append(encoding[next_char])

num_sentences = len(X_data)
print("Sliced our corpus into {0} sentences of length {1}"
      .format(num_sentences, sentence_length))
```

```
Sliced our corpus into 1115374 sentences of length 20
```

```
print(X_data[0])
```

```
[18, 47, 56, 57, 58, 1, 15, 47, 58, 47, 64, 43, 52, 10, 0, 14, 4
```

```
print([decoding[idx] for idx in X_data[0]])
print(decoding[y_data[0]])
```

```
['F', 'i', 'r', 's', 't', ' ', 'C', 'i', 't', 'i', 'z', 'e', 'n'
e
```

Prepare the inputs and outputs and train

Vectorise the data and target into one-hot representation.

```
# Vectorize our data and labels. We want everything in one-hot.
X = np.zeros((num_sentences, sentence_length, num_chars),dtype=bool)
y = np.zeros((num_sentences, num_chars), dtype=bool)
for i, sentence in enumerate(X_data):
    for t, encoded_char in enumerate(sentence):
        X[i, t, encoded_char] = 1
    y[i, y_data[i]] = 1
```

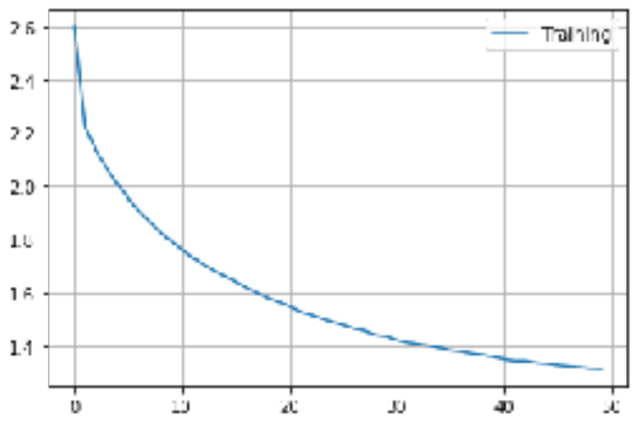
Sanity check on the tensor sizes.

```
# Double check our vectorized data before we sink hours into fitting
a model
print("Sanity check y. Dimension: {0} # Sentences: {1} Characters in
corpus: {2}"
      .format(y.shape, num_sentences, len(chars)))
print("Sanity check X. Dimension: {0} Sentence length: {1}"
      .format(X.shape, sentence_length))
```

```
Sanity check y. Dimension: (1115374, 65) # Sentences: 1115374 Characters in corpus: 65
Sanity check X. Dimension: (1115374, 20, 65) Sentence length: 20
```

Train the network

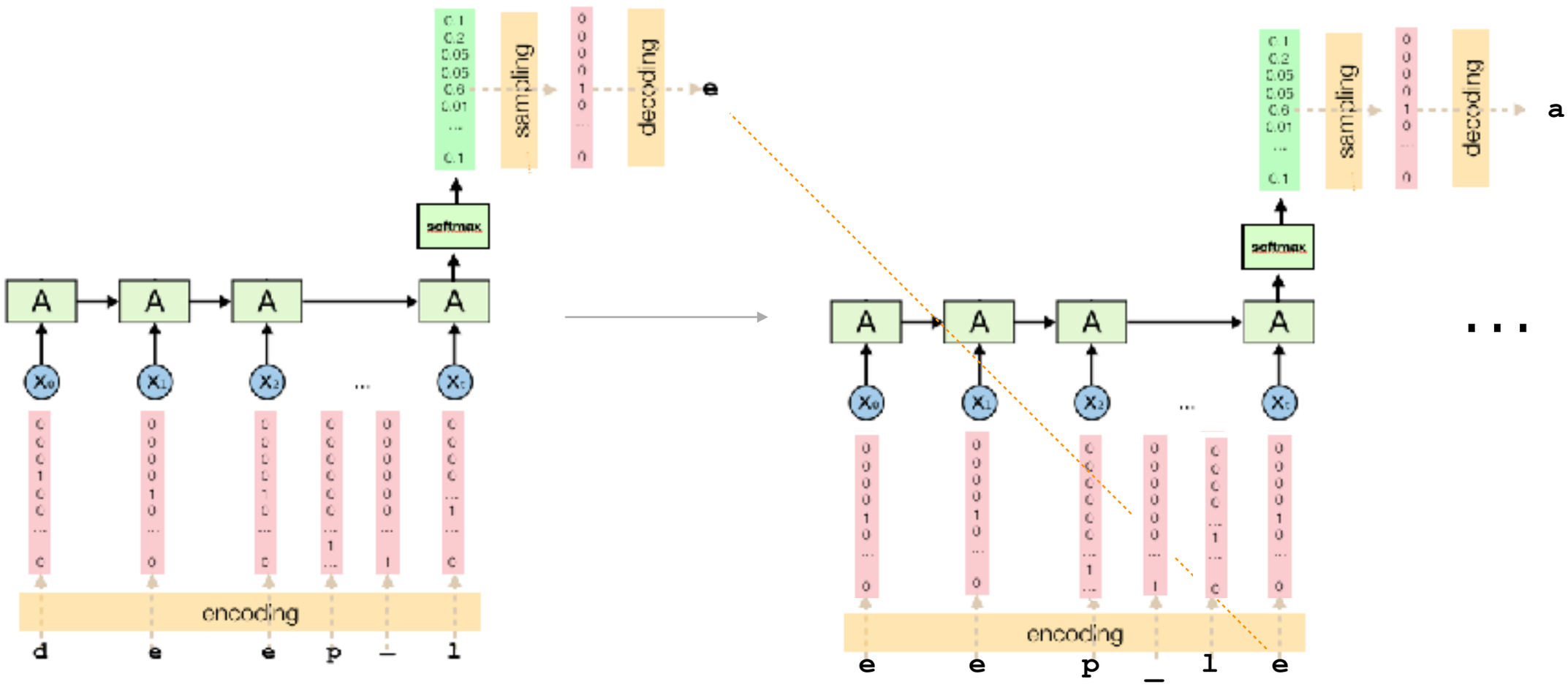
```
#training time
log = model.fit(X, y, epochs=50, batch_size=128)
plt.plot(log.history['loss'], label='Training')
plt.legend()
plt.grid()
```




```
Train on 111538 samples
Epoch 1/50
111538/111538 [==============================] - 17s 156us/sample - loss: 2.5987 - accuracy: 0.2961
Epoch 2/50
111538/111538 [==============================] - 16s 143us/sample - loss: 2.2253 - accuracy: 0.3647
Epoch 3/50
111538/111538 [==============================] - 16s 142us/sample - loss: 2.1340 - accuracy: 0.3854
Epoch 4/50
111538/111538 [==============================] - 15s 139us/sample - loss: 2.0681 - accuracy: 0.3983
```

Using the trained network

1. Define a seed, e.g. `[d, e, e, p,  , l]`
2. Predict the output probabilities and sample from it the next char: e.g. `'e'`
3. Shift the input: remove 1st char and append prediction, e.g. `[e, e, p,  , l, e]`, go back to 2



Using the trained network

```python
def make_seed(seed_phrase=""):
        if seed_phrase:
            phrase_length = len(seed_phrase)
            pattern = ""
            for i in range (0, sentence_length):
                pattern += seed_phrase[i % phrase_length]
        else:
            seed = randint(0, corpus_length - sentence_length)
            pattern = corpus[seed:seed + sentence_length]

        return pattern

seed_pattern = make_seed("In the early morning, the flower is
shining”)

X = np.zeros((1, sentence_length, num_chars), dtype=float)
for i, character in enumerate(seed_pattern):
    X[0, i, encoding[character]] = 1

generated_text = ""
for i in range(500):
    output_prob = model.predict(X, verbose=0)[0]
    prediction = np.random.choice(num_chars, p = output_prob )
    generated_text += decoding[prediction]
    activations = np.zeros((1, 1, num_chars), dtype=bool)
    activations[0, 0, prediction] = 1
    #now remove first char and glue the predicted one
    X = np.concatenate((X[:, 1:, :], activations), axis=1)
print(generated_text)
```

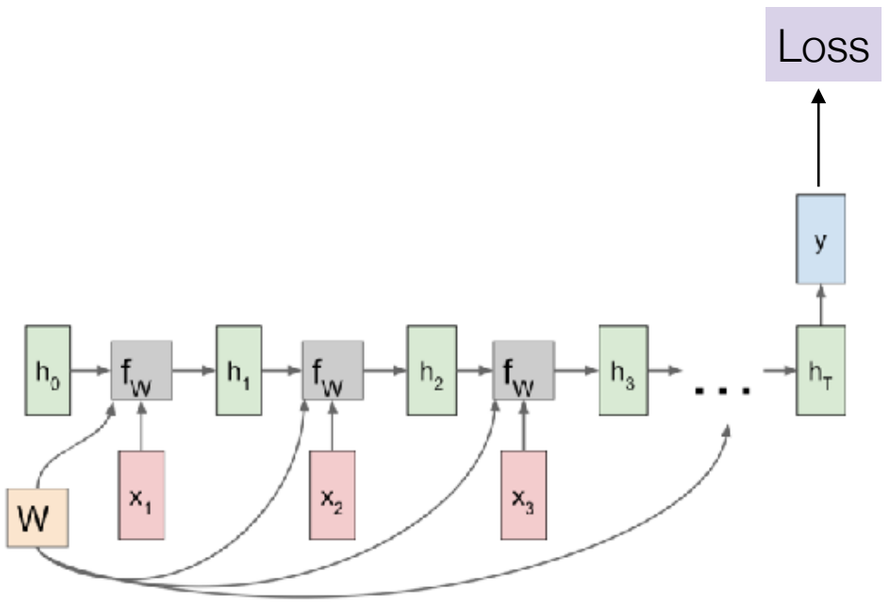
```
KING RICHARD III:
Ay, ank you, Inchine; are well welp you? lake so
busting and to the enmate by a marriof a
bato, my fearuf
Than a great Jurn thesdevtive;
For I know nothing?.

ToUMELLO:
Heak's I sceets! Geor my denther to a fait.

ROMEO:
Cationd,--tile,
And fair mever hew rents of wellifol' thate and to the
bast word
Your presons!
We would bring fots father's belo:
Unllow, man's pay of Somelf.

LUCHASCETTBUCHASTIFFFROMESLoO:
I absed both rearps meant titer tor
```

predict() gives a tensor of shape (1, 65) with 1 being the size of the batch. For that we use [0]to remove the batch dim and get a vector of size 65.

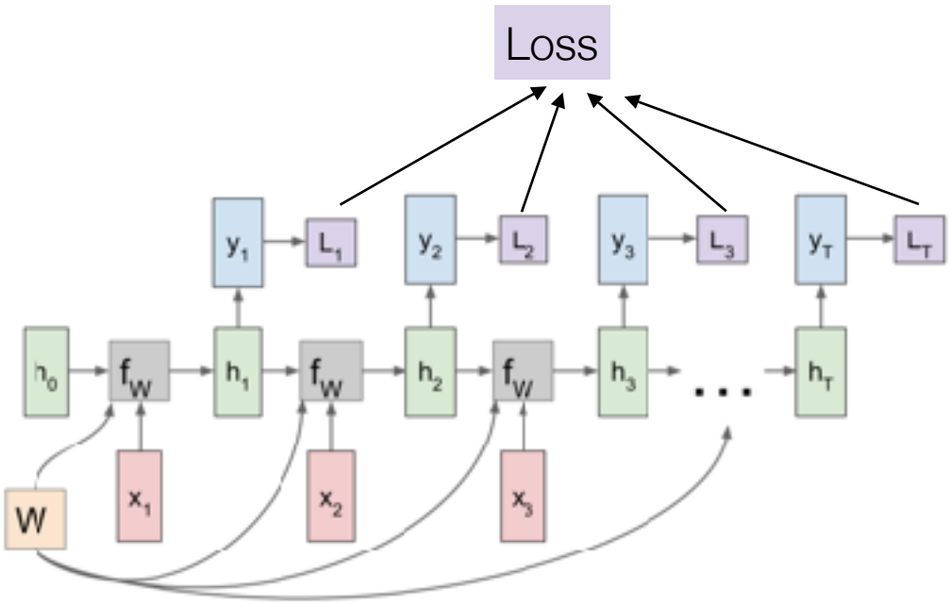
We randomly pick the winner char respecting the output distribution.

Shift the input with the new generated char and begin again.

Many to many approach



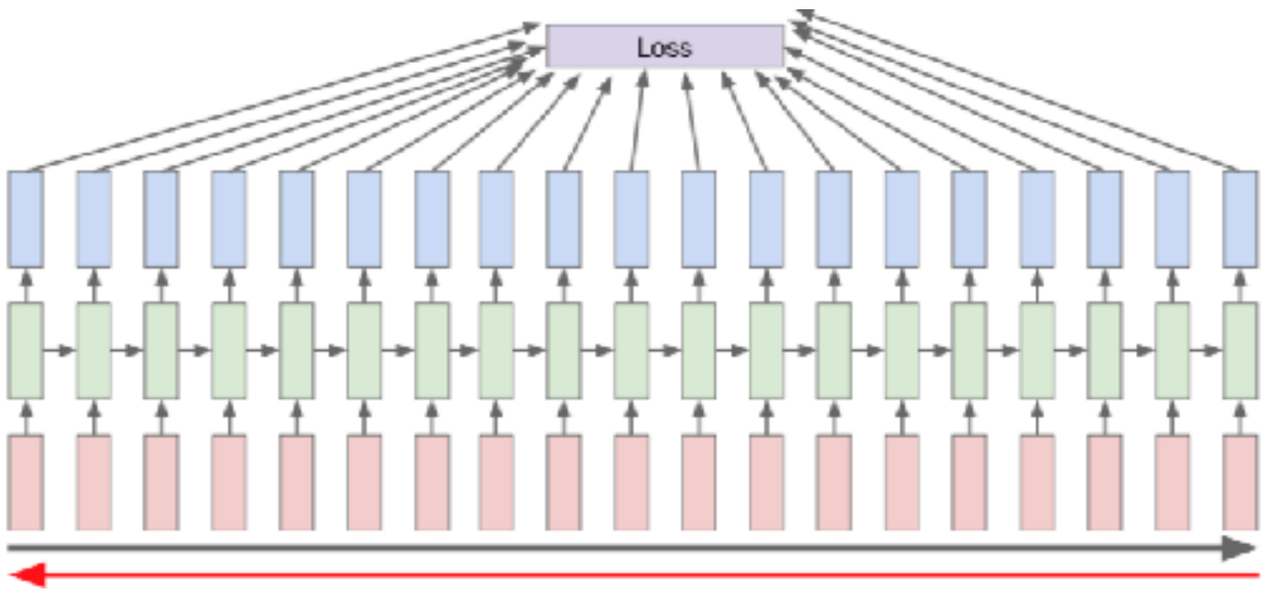
Many to one



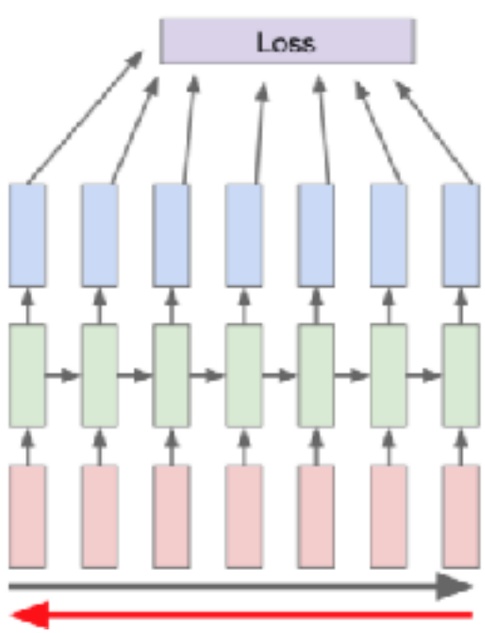
Many to many

- The difference is mainly at training time where the RNN cells are emitting an output at each step.
- The loss is computed from all the outputs, versus 1 output on the many to one approach
- The output layer is “time distributed” to the whole sequence

Truncated back propagation through time



In theory: forward through the entire sequence to compute loss, then backward through the entire sequence to compute gradient. But not really feasible if the sequence is long.



In practice: run forward and backward through chunks of the sequence instead of whole sequence

Define the many-to-many model in `Keras`

SimpleRNN layer:

- **`return_sequences=True`:** to tell the network to give an output at each step of the sequence.

We wrap the Dense output layer with a TimeDistributed layer to "distribute the output over each step of the sequence".

```
# Define our model
model = keras.models.Sequential()
model.add(keras.layers.SimpleRNN(256, input_shape=(sentence_length, num_chars),
                                 return_sequences=True))
model.add(keras.layers.TimeDistributed(keras.layers.Dense(num_chars,
                                                          activation='softmax')))
model.compile(loss='categorical_crossentropy', optimizer='adam',
              metrics=['accuracy'])
model.summary()
```

```
Model: "sequential"
_________________________________________________________________
Layer (type)                 Output Shape              Param #
=================================================================
simple_rnn (SimpleRNN)       (None, 100, 256)          82432
_________________________________________________________________
time_distributed (TimeDistri (None, 100, 65)           16705
=================================================================
Total params: 99,137
Trainable params: 99,137
Non-trainable params: 0
_________________________________________________________________
```

$$\mathbf{w}_h : 256 \times 256 = 65536$$
$$\mathbf{w}_x : 256 \times 65 = 16640$$
$$\mathbf{b}_h : 256 \times 1 = 256$$

82432

$$256 \times 65 + 65 = 16705$$

unique char = 65

Data preparation is a bit different for y

`sentence_length` is the length of the sequence and `skip` defines the stride we do on the input sequence

Chop the corpus into sentences of length 20. Take 20 chars shifted of 1 as target.

```
# chop up our data into X and y, slice into roughly
# (num_chars / skip) overlapping 'sentences' of length
# sentence_length, and encode the chars
sentence_length = 20
skip = 1
X_data = []
y_data = []
for i in range (0, len(corpus) - sentence_length, skip):
    sentence = corpus[i:i + sentence_length]
    next_char = corpus[i+1:i+1 + sentence_length]
    X_data.append([encoding[char] for char in sentence])
    y_data.append([encoding[char] for char in next_char])

num_sentences = len(X_data)
print("Sliced our corpus into {0} sentences of length {1}"
      .format(num_sentences, sentence_length))
```

```
Sliced our corpus into 1115374 sentences of length 20
```

A given input sequence is encoded into the corresponding char index defined in the encoding dictionary.

```
print(X_data[0])
```

```
[18, 47, 56, 57, 58, 1, 15, 47, 58, 47, 64, 43, 52, 10, 0, 14, 4
```

An example of x and y.

```
print([decoding[idx] for idx in X_data[0]])
print([decoding[idx] for idx in y_data[0]])
```

```
['F', 'i', 'r', 's', 't', ' ', 'C', 'i', 't', 'i', 'z', 'e', 'n', '
:', '\n', 'B', 'e', 'f', 'o', 'r']
['i', 'r', 's', 't', ' ', 'C', 'i', 't', 'i', 'z', 'e', 'n', ':', '
\n', 'B', 'e', 'f', 'o', 'r', 'e']
```

y is then a tensor of the same dimensions as x: (1115374, 20, 65)

```
# Vectorize our data and labels. We want everything in one-hot.
X = np.zeros((num_sentences, sentence_length, num_chars), dtype=bool)
y = np.zeros((num_sentences, sentence_length, num_chars), dtype=bool)
for i, sentence in enumerate(X_data):
    for t, encoded_char in enumerate(sentence):
        X[i, t, encoded_char] = 1
for i, sentence in enumerate(y_data):
    for t, encoded_char in enumerate(sentence):
        y[i, t, encoded_char] = 1
```

Using the trained network (as for many-to-one)

```
def make_seed(seed_phrase=""):
        if seed_phrase:
            phrase_length = len(seed_phrase)
            pattern = ""
            for i in range (0, sentence_length):
                pattern += seed_phrase[i % phrase_length]
        else:
            seed = randint(0, corpus_length - sentence_length)
            pattern = corpus[seed:seed + sentence_length]

        return pattern

seed_pattern = make_seed("In the early morning, the flower is
shining”)

X = np.zeros((1, sentence_length, num_chars), dtype=float)
for i, character in enumerate(seed_pattern):
    X[0, i, encoding[character]] = 1

generated_text = ""
for i in range(500):
    output_prob = model.predict(X, verbose=0)[0][-1]
    prediction = np.random.choice(num_chars, p = output_prob )
    generated_text += decoding[prediction]
    activations = np.zeros((1, 1, num_chars), dtype=bool)
    activations[0, 0, prediction] = 1
    #now remove first char and glue the predicted one
    X = np.concatenate((X[:, 1:, :], activations), axis=1)
print(generated_text)
```

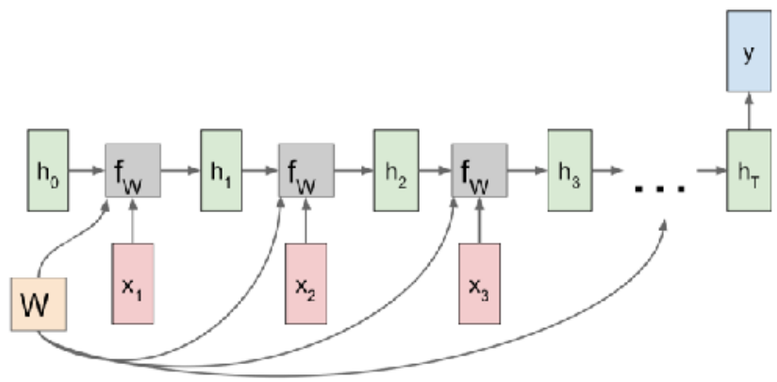
```
Speak that poor cham have me godds; I
is enough to pernician footingbade of fear they ashing-
cover'd were no neart-grant.

SICINIUS:
For is't nee gratten all his house off lords, an is little.

PRINCE:
Me me, to be upon the murgs to souls with the life and
prinent I huspies sun, you do releep the law both as King
Livility.

KING RICHARD III:
Stay all-wearthe of traitor him;
For soever
By a news, how he livest heart…
```

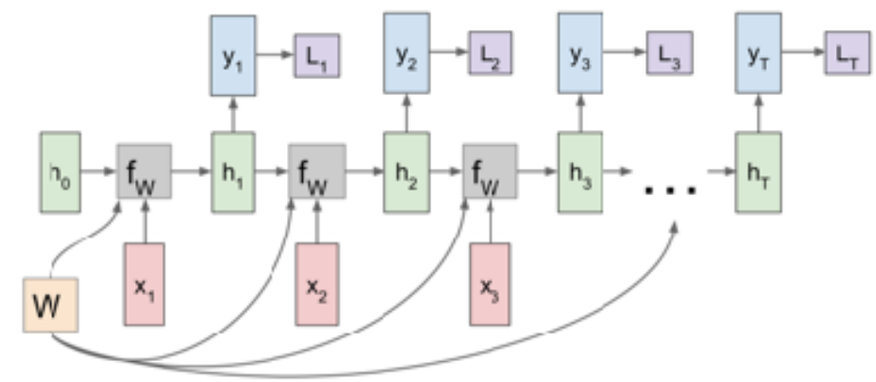
predict() gives a tensor of shape (1, 20, 65) with 1 being the size of the batch and 20 the prediction at each step. For that we use [0][-1] to remove the batch dim and get the last prediction : a vector of size 65.

We randomly pick the winner char respecting the output distribution.

Shift the input with the new generated char and begin again.

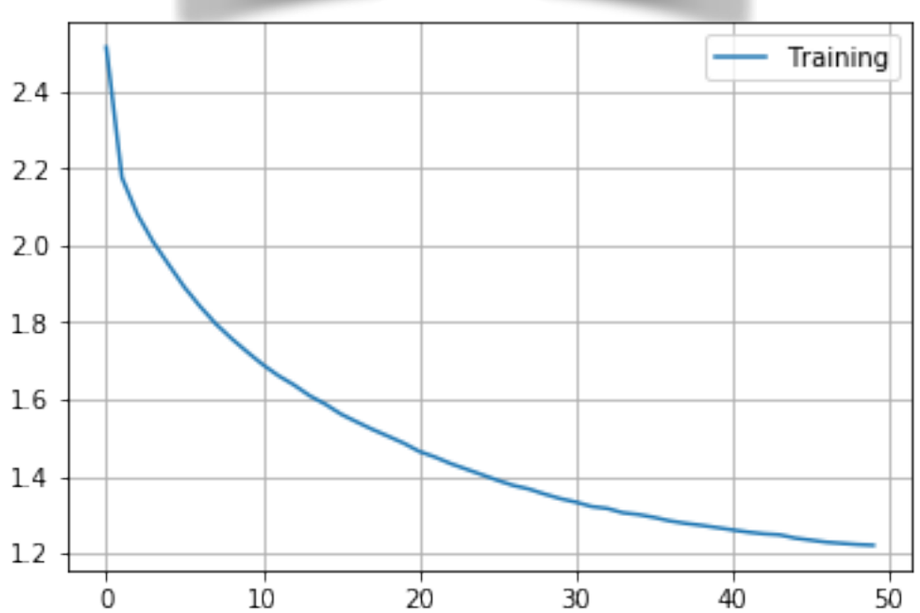
Many-to-one vs many-to-many

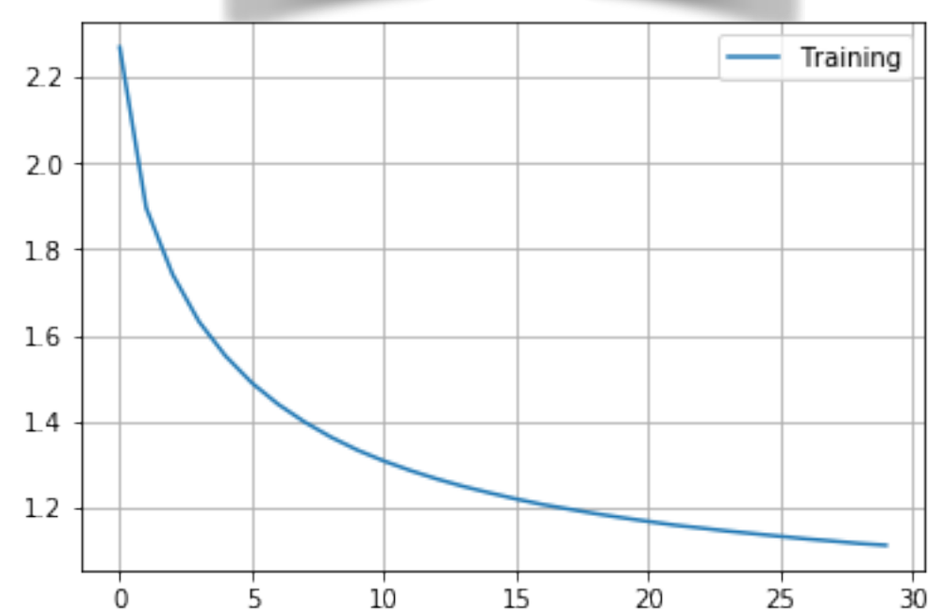




Many to one
sentence_length = 20
skip = 1
SimpleRNN, h_dim=256

Many to many
sentence_length = 20
skip = 1
SimpleRNN, h_dim=256

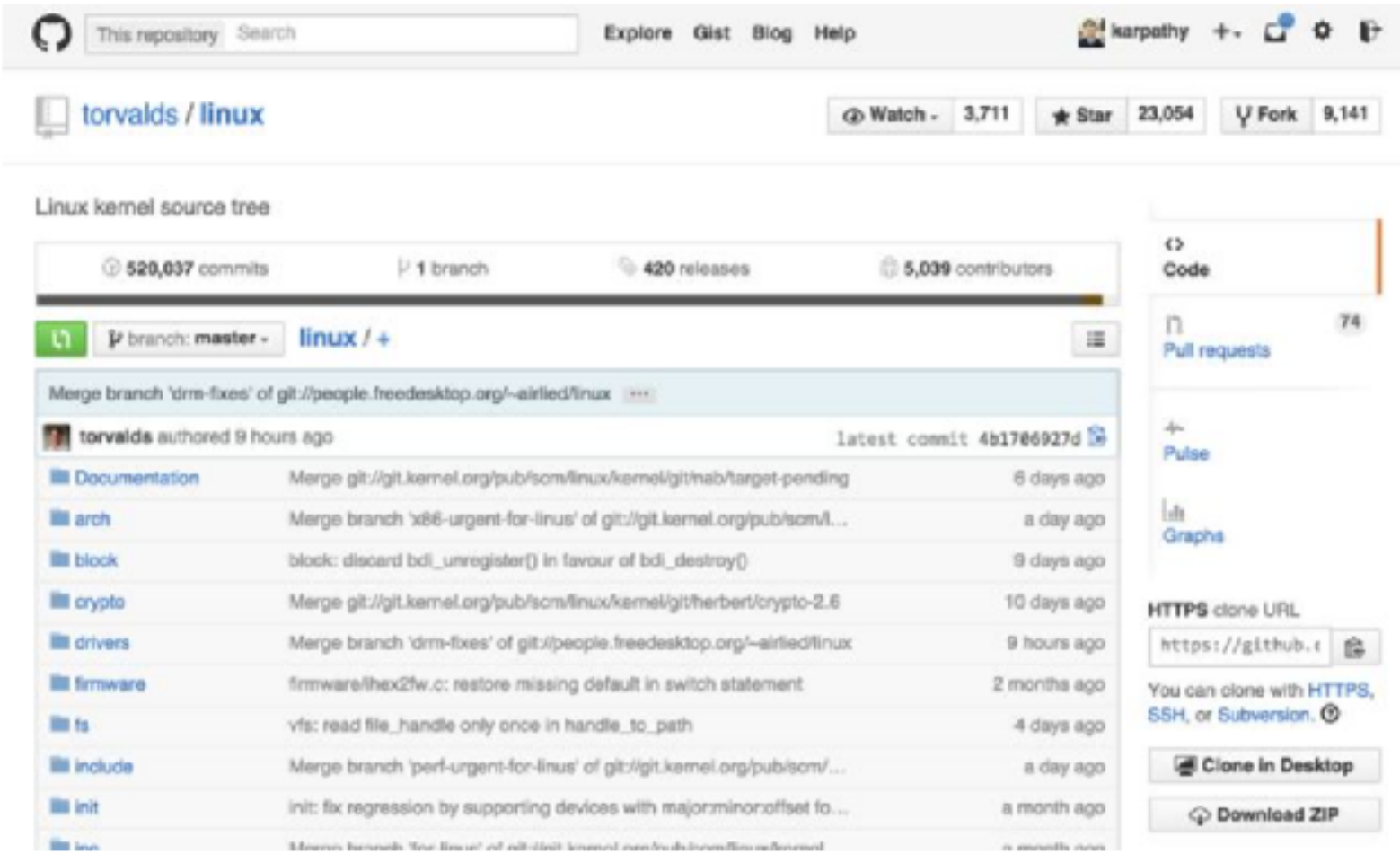




Better loss is reached in less epochs, which seems reasonable as training is done at each step of the recurrence with the many-to-many configuration.

Other examples - C code generation

- See e.g. http://karpathy.github.io/2015/05/21/rnn-effectiveness/
- Train a RNN on the source code of Linux kernel



Other examples - C code generation

```c
/*
 *  Copyright (c) 2006-2010, Intel Mobile Communications.  All rights reserved.
 *
 *   This program is free software; you can redistribute it and/or modify it
 * under the terms of the GNU General Public License version 2 as published by
 * the Free Software Foundation.
 *
 *        This program is distributed in the hope that it will be useful,
 * but WITHOUT ANY WARRANTY; without even the implied warranty of
 *   MERCHANTABILITY or FITNESS FOR A PARTICULAR PURPOSE.  See the
 *
 *  GNU General Public License for more details.
 *
 *   You should have received a copy of the GNU General Public License
 *    along with this program; if not, write to the Free Software Foundation,
 *  Inc., 675 Mass Ave, Cambridge, MA 02139, USA.
 */

#include <linux/kexec.h>
#include <linux/errno.h>
#include <linux/io.h>
#include <linux/platform_device.h>
#include <linux/multi.h>
#include <linux/ckevent.h>

#include <asm/io.h>
#include <asm/prom.h>
#include <asm/e820.h>
#include <asm/system_info.h>
#include <asm/setew.h>
#include <asm/pgproto.h>
```

```c
static void do_command(struct seq_file *m, void *v)
{
  int column = 32 << (cmd[2] & 0x80);
  if (state)
    cmd = (int)(int_state ^ (in_8(&ch->ch_flags) & Cmd) ? 2 : 1);
  else
    seq = 1;
  for (i = 0; i < 16; i++) {
    if (k & (1 << 1))
      pipe = (in_use & UMXTHREAD_UNCCA) +
        ((count & 0x00000000fffffff8) & 0x000000f) << 8;
    if (count == 0)
      sub(pid, ppc_md.kexec_handle, 0x20000000);
    pipe_set_bytes(i, 0);
  }
  /* Free our user pages pointer to place camera if all dash */
  subsystem_info = &of_changes[PAGE_SIZE];
  rek_controls(offset, idx, &soffset);
  /* Now we want to deliberately put it to device */
  control_check_polarity(&context, val, 0);
  for (i = 0; i < COUNTER; i++)
    seq_puts(s, "policy ");
}
```

Pointers to other generative models

- Variational Auto-Encoders - VAR
- Generative Adversarial Networks - GAN
- See for example
  - Stanford Class CS231n Convolutional Neural Networks for Visual Recognition
    - Lecture 13 on **Generative Models** from https://youtu.be/5WoItGTWV54
  - Stanford Class CS236 Deep Generative Models
    - https://deepgenerativemodels.github.io

Wrap-up

- **Generative RNNs** are able to generate new consistent data from a seed.
  - Consistent = respecting temporal or spatial “structures” that have been learned from the input space
  - Two approaches: many-to-one or many-to-many
  - The many-to-many approach needs a distribution of the output through time and a truncated back-propagation through time